Package Structure and Notation

The diagram in Figure 22-1 shows a possible package structure for the payroll application. We will address the appropriateness of this structure later. For now, we will confine ourselves to how such a structure is documented and used.

See page 481 for a description of the UML notation for packages. By convention, package diagrams are drawn with the dependencies pointing downwards. Packages at the top are dependent. Packages at the bottom are depended on.

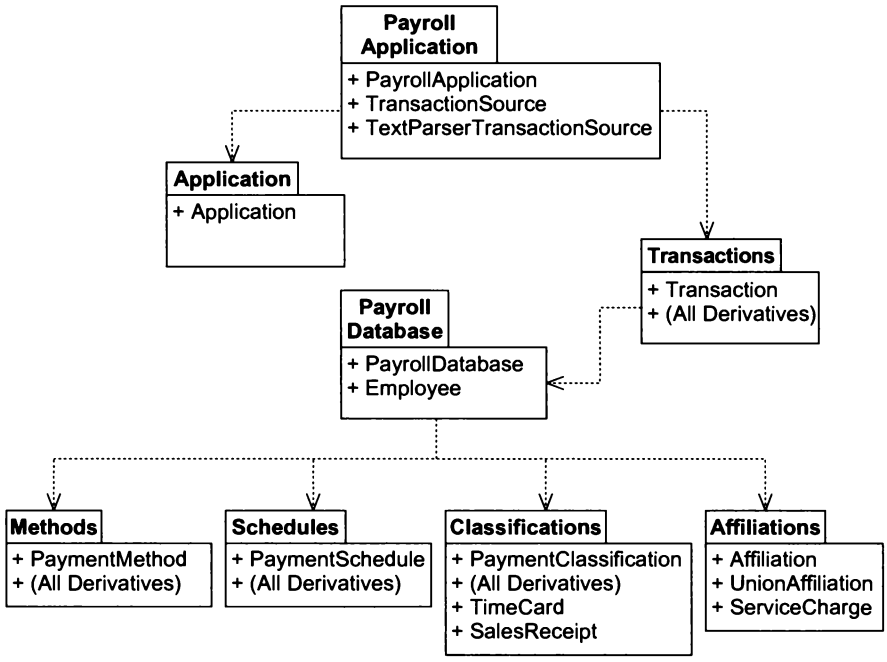


Figure 22-1 Possible Payroll Package Diagram

Figure 22-1 has divided the payroll application into eight packages. The `PayrollApplication` package contains the `PayrollApplication` class and the `TransactionSource` and `TextParserTransactionSource` classes. The `Transactions` package contains the complete `Transaction`-class hierarchy. The constituents of the other packages should be clear by carefully examining the diagram.

The dependencies should also be clear. The `PayrollApplication` package depends on the `Transactions` package because the `PayrollApplication` class calls the `Transaction::Execute` method. The `Transactions` package depends on the `PayrollDatabase` package because each of the many derivatives of `Transaction` communicate directly with the `PayrollDatabase` class. The other dependencies are likewise justifiable.

What criteria did I use to group these classes into packages? I simply stuck the classes that look like they belonged together into the same packages. As we learned in Chapter 20, this is probably not a good idea.

Consider what happens if we make a change to the `Classifications` package. This change will force a recompilation and retest of the `EmployeeDatabase` package, and well it should. But it will also force a recompilation and retest of the `Transactions` package. Certainly the `ChangeClassificationTransaction` and its three derivatives from Figure 19-3 *should* be recompiled and retested, but why should the others be recompiled and retested?

Technically, those other transactions don't need recompilation and retest. However, if they are part of the `Transactions` package, and if the package is going to be rereleased to deal with the changes to the

Classifications package, then it could be viewed as irresponsible not to recompile and retest the package as a whole. Even if all the transactions aren't recompiled and retested, the package itself must be rereleased and redeployed, and then all of its clients will require revalidation at the very least, and probably recompilation.

The classes in the Transactions package do not share the same closure. Each one is sensitive to its own particular changes. The ServiceChargeTransaction is open to changes to the ServiceCharge class, whereas the TimeCardTransaction is open to changes to the TimeCard class. In fact, as the diagram in Figure 22-1 implies, some portion of the Transactions package is dependent on nearly every other part of the software. Thus, this package suffers a very high rate of release. Every time something is changed anywhere below, the Transactions package will have to be revalidated and rereleased.

The PayrollApplication package is even more sensitive: any change to any part of the system will affect this package, so its release rate must be enormous. You might think that this is inevitable—that as one climbs higher up the package-dependency hierarchy, the release rate must increase. Fortunately, however, this is not true, and avoiding this symptom is one of the major goals of OOD.

Applying the Common-Closure Principle (CCP)

Consider Figure 22-2. This diagram groups the classes of the payroll application together according to their closure. For example, the PayrollApplication package contains the PayrollApplication and TransactionSource classes. These two classes both depend on the abstract Transaction class, which is in the

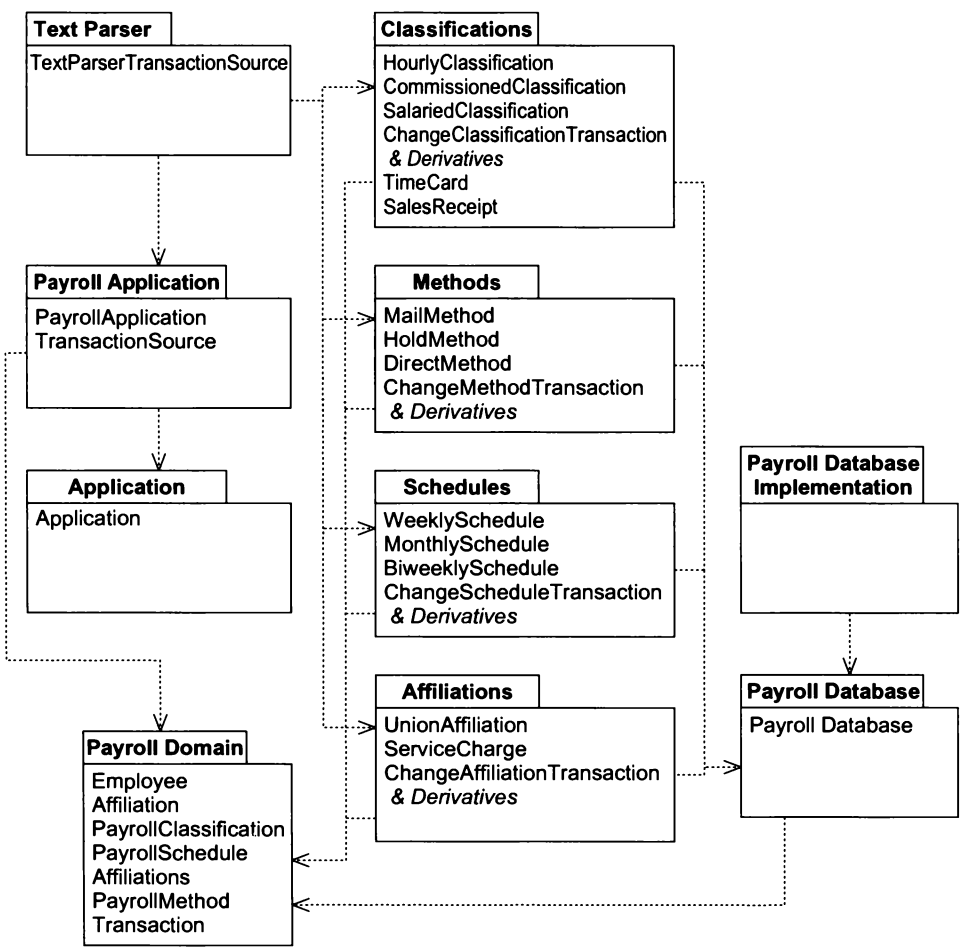


Figure 22-2 A Closed Package Hierarchy for the Payroll Application

PayrollDomain package. Note that the TextParserTransactionSource class is in another package that depends on the abstract PayrollApplication class. This creates an upside-down structure in which the details depend on the generalities, and the generalities are independent. This conforms to the DIP.

The most striking case of generality and independence is the PayrollDomain package. This package contains the *essence* of the whole system, yet it depends on nothing! Examine this package carefully. It contains Employee, PaymentClassification, PaymentMethod, PaymentSchedule, Affiliation, and Transaction. This package contains all of the major abstractions in our model, yet it has no dependencies. Why? Because nearly all of the classes it contains are abstract.

Consider the Classifications package, which contains the three derivatives of PaymentClassification. It also contains the ChangeClassificationTransaction class and its three derivatives, along with TimeCard and SalesReceipt. Notice that any change made to these nine classes is isolated; other than TextParser, no other package is affected! Such isolation also holds for the Methods package, the Schedules package, and the Affiliations package. This is quite a bit of isolation.

Notice that the bulk of the executable code is in packages that have few or no dependents. Since almost nothing depends on them, we call them *irresponsible*. The code within those packages is tremendously flexible; it can be changed without affecting many other parts of the project. Notice also that the most general packages of the system contain the least amount of executable code. These packages are heavily depended on, but depend on nothing. Since many packages depend on them, we call them *responsible*, and since they don't depend on anything, we call them *independent*. Thus, the amount of responsible code (i.e., code in which changes would affect lots of other code) is very small. Moreover, that small amount of responsible code is also independent, which means that no other modules will induce it to change. This upside-down structure, with highly independent and responsible generalities at the bottom, and highly irresponsible and dependent details at the top, is the hallmark of object-oriented design.

Contrast Figure 22-1 with Figure 22-2. Notice that the details at the bottom of Figure 22-1 are independent and highly responsible. This is the wrong place for details! Details should depend on the major architectural decisions of the system and should not be depended on. Notice also that the generalities, the packages that define the architecture of the system, are irresponsible and highly dependent. Thus, the packages that define the architectural decisions depend on, and are thus constrained by, the packages that contain the implementation details. This is a violation of the SAP. It would be better if the architecture constrained the details!

Applying the Reuse–Release Equivalency Principle (REP)

What portions of the payroll application can we reuse? If another division of our company wanted to reuse our payroll system, but they had a completely different set of policies, they could not reuse Classifications, Methods, Schedules, or Affiliations. However, they could reuse PayrollDomain, PayrollApplication, Application, PayrollDatabase, and possibly PDImplementation. On the other hand, if another department wanted to write software that analyzed the current employee database, they could reuse PayrollDomain, Classifications, Methods, Schedules, Affiliations, PayrollDatabase, and PDImplementation. In each case, the granule of reuse is a package.

Seldom, if ever, would only a single class from a package be reused. The reason is simple: the classes within a package should be cohesive. That means that they depend on one another and cannot be easily or sensibly separated. It would make no sense, for example, to use the Employee class without using the PaymentMethod class. In fact, in order to do so, you would have to modify the Employee class so that it did not contain a PaymentMethod class. Certainly we don't want to support the kind of reuse that forces us to modify the reused components. Therefore, the granule of reuse is the package. This gives us another cohesion criterion to employ when trying to group classes into packages: not only should the classes be closed together, they should also be reusable together in conformance with the REP.

Consider again our original package diagram in Figure 22-1. The packages that we might like to reuse, like Transactions or PayrollDatabase, are not easily reusable because they drag along a lot of extra baggage. The PayrollApplication package is horribly dependent (it depends on everything). If we wanted to create a new payroll application that used a different set of schedule, method, affiliation, and classification policies, we would not be able to use this package as a whole. Instead, we would have to take individual classes from PayrollApplication, Transactions, Methods, Schedules, Classifications, and Affiliations. By disassembling the packages in this way, we destroy their release structure. We cannot say that Release 3.2 of PayrollApplication is reusable.

Figure 22-1 violates the CRP. Thus, having accepted the reusable fragments of our various packages, the reuser will be faced with a difficult management problem: he will not be able to depend on our release structure. A new release of Methods affects him because he is reusing the PaymentMethod class. Most of the time, the changes will be to classes that he is not reusing, yet he must still track our new release number and probably recompile and retest his code.

This can be so difficult to manage that the reuser's most likely strategy will be to make a copy of the reusable components and evolve that copy separately from ours. This is not reuse. The two pieces of code will become different and will require independent support, effectively doubling the support burden.

These problems are not exhibited by the structure in Figure 22-2. The packages in that structure are easier to reuse. PayrollDomain does not drag along much baggage. It is reusable independently of any of the derivatives of PaymentMethod, PaymentClassification, PaymentSchedule, etc.

The astute reader will notice that the package diagram in Figure 22-2 does not completely conform to the CRP. Specifically, the classes within PayrollDomain do not form the smallest reusable unit. The Transaction class does not need to be reused with the rest of the package. We could design many applications that access the Employee and its fields, but never use a Transaction.

This suggests a change to the package diagram, as shown in Figure 22-3. This separates the transactions from the elements that they manipulate. For example, the classes in the MethodTransactions package manipulate the classes in the Methods package. We have moved the Transaction class into a new package named TransactionApplication, which also contains TransactionSource and a class named TransactionApplication. These three form a reusable unit. The PayrollApplication class has now become the grand unifier. It contains the main program and also a derivative of TransactionApplication called PayrollApplication, which ties the TextParserTransactionSource to the TransactionApplication.

These manipulations have added yet another layer of abstraction to the design. The TransactionApplication package can now be reused by any application that obtains Transactions from a TransactionSource and then Executes them. The PayrollApplication package is no longer reusable, since it is extremely dependent. However, the TransactionApplication package has taken its place, and is more general. Now we can reuse the PayrollDomain package without any Transactions.

This certainly improves the reusability and maintainability of the project, but at the cost of five extra packages and a more complex dependency architecture. The value of the trade-off depends on the type of reuse that we might expect and the rate at which we expect the application to evolve. If the application remains very stable, and few clients reuse it, then this change is overkill. On the other hand, if many applications will reuse this structure, or we expect the application to experience many changes, then the new structure is superior—it's a judgment call; and it should be driven by data rather then speculation. It is best to start simple and grow the package structure as necessary. Package structures can always be made more elaborate if necessary.

Coupling and Encapsulation

Just as the coupling among classes is managed by encapsulation boundaries in Java and C++, so the couplings among packages can be managed by the export adornments of the UML.

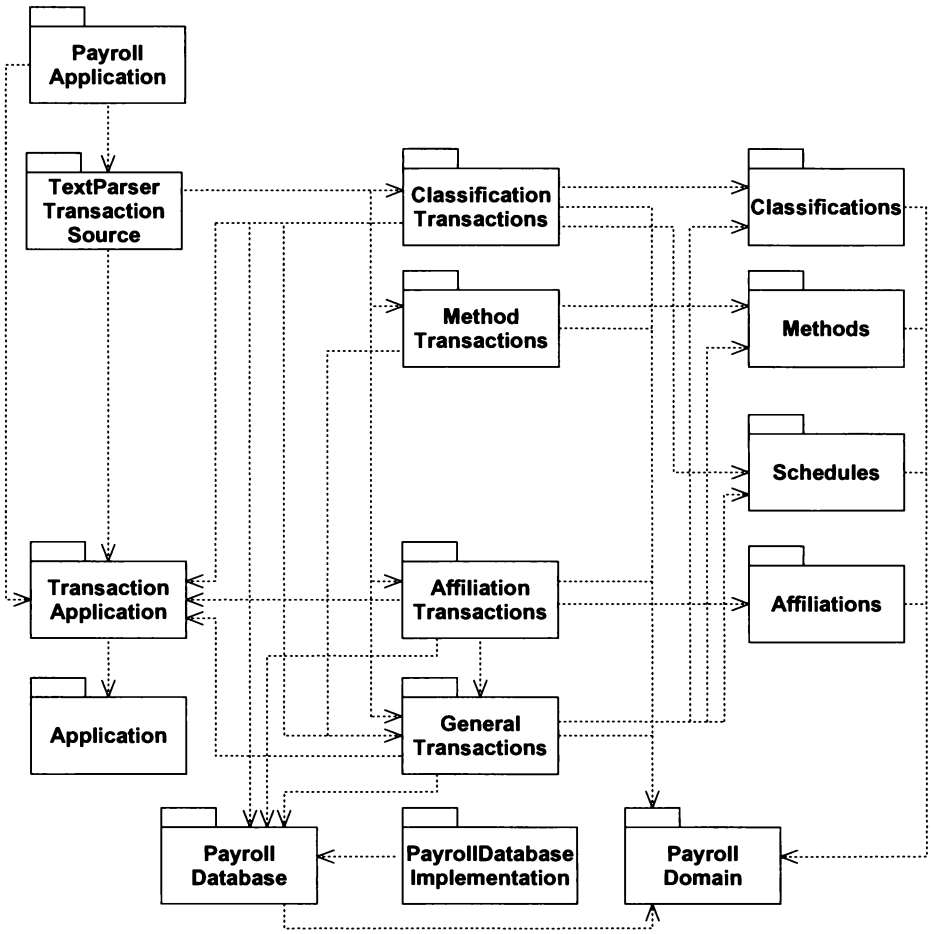


Figure 22-3 Updated Payroll Package Diagram

If a class within one package is to be used by another package, that class must be exported. In UML, classes are exported by default, but we may adorn a package to denote that certain classes should not be exported. Figure 22-4, a blowup of the `Classifications` package, shows that the three derivatives of `PaymentClassification` are exported, but that `TimeCard` and `SalesReceipt` are not. This means that other packages will not be able to use `TimeCard` and `SalesReceipt`; they are private to the `Classifications` package.

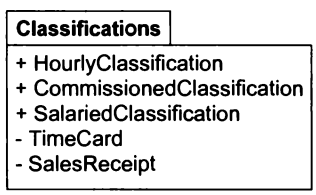


Figure 22-4 Private Classes in Classifications Package

We may want to hide certain classes within a package to prevent afferent couplings. `Classifications` is a very detailed package that contains the implementations of several payment policies. In order to keep this package on the main sequence, we want to limit its afferent couplings, so we hide the classes that other packages don't need to know about.

`TimeCard` and `SalesReceipt` are good choices for private classes. They are implementation details of the mechanisms for calculating an employee's pay. We want to remain free to alter these details, so we need to prevent anyone else from depending on their structure.

A quick glance at Figure 19-7 through Figure 19-10 and Listing 19-15 (page 213 through page 217) shows that the `TimeCardTransaction` and `SalesReceiptTransaction` classes already depend on `TimeCard` and `SalesReceipt`. We can easily resolve this problem, however, as shown in Figure 22-5 and Figure 22-6.

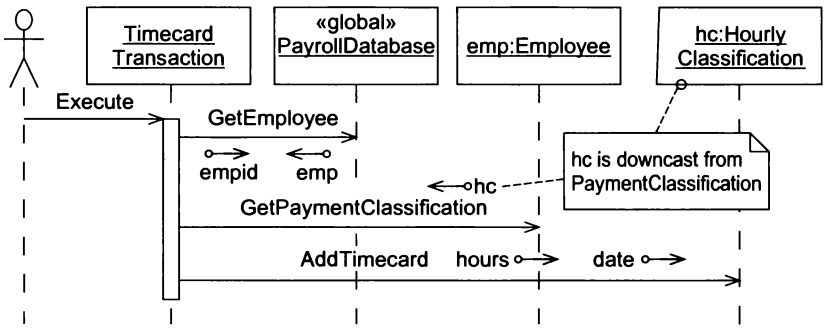


Figure 22-5 Revision to `TimeCardTransaction` to Protect `TimeCard` Privacy

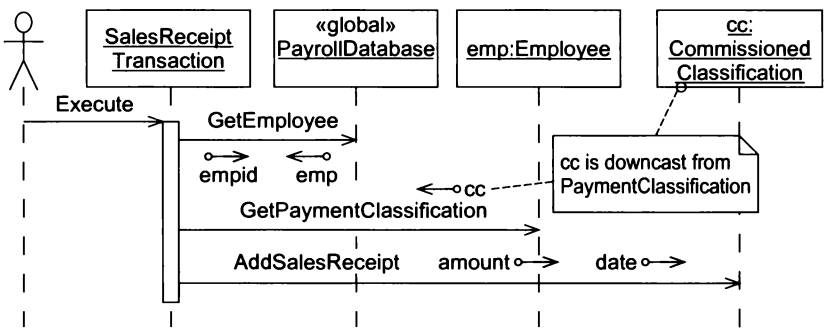


Figure 22-6 Revision to `SalesReceiptTransaction` to Protect `SalesReceipt` Privacy

Metrics

As we showed in Chapter 20, we can quantify the attributes of cohesion, coupling, stability, generality, and conformance to the main sequence with a few simple metrics. But why should we want to? To paraphrase Tom DeMarco: You can't manage what you can't control, and you can't control what you don't measure.[1] To be effective software engineers or software managers, we must be able to control software development practice. If we don't measure it, however, we will never have that control.

By applying the heuristics described below and calculating some fundamental metrics about our object-oriented designs, we can begin to correlate those metrics with measured performance of the software and of the teams that develop it. The more metrics we gather, the more information we will have, and the more control we will eventually be able to exert.

The metrics below have been successfully applied to a number of projects since 1994. There are several automatic tools that will calculate them for you, and they are not hard to calculate by hand. It is also not hard to write a simple shell, python, or ruby script to walk through your source files and calculate them.[2]

1. [DeMarco82], p. 3.
2. For an example of a shell script, you can download `depend.sh` from the freeware section of `www.objectmentor.com`., or take a look at JDepend at www.clarkware.com.

- **(*H*) Relational Cohesion**. One aspect of the cohesion of a package can be represented as the average number of internal relationships per class. Let R be the number of class relationships that are internal to the package (i.e., that do not connect to classes outside the package). Let N be the number of classes within the package). The extra 1 in the formula prevents $H = 0$ when $N = 1$. It represents the relationship that the package has to all its classes.

$$H = \frac{R + 1}{N}$$

- **(C_a) Afferent coupling** can be calculated as the number of classes from other packages that depend on the classes within the subject package. These dependencies are class relationships, such as inheritance and association.
- **(C_e) Efferent coupling** can be calculated as the number of classes in other packages that the classes in the subject package depend on. As before, these dependencies are class relationships.
- **(*A*) Abstractness** or **Generality** can be calculated as the ratio of the number of abstract classes (or interfaces) in the package to the total number of classes (and interfaces) in the package.[3] This metric ranges from 0 to 1.

$$A = \frac{\text{Abstract Classes}}{\text{Total Classes}}$$

- **(*I*) Instability** can be calculated as the ratio of efferent coupling to total coupling. This metric also ranges from 0 to 1.

$$I = \frac{C_e}{C_e + C_a}$$

- **(*D*) Distance from the Main Sequence.** The main sequence is idealized by the line $A + I = 1$. The formula for D calculates the distance of any particular package from the main sequence. It ranges from ~.7 to 0;[4] the closer to 0, the better.

$$D = \frac{|A + I - 1|}{\sqrt{2}}$$

- **(*D′*) Normalized Distance from the Main Sequence**. This metric represents the D metric normalized to the range [0,1]. It is perhaps a little more convenient to calculate and to interpret. The value 0 represents a package that is coincident with the main sequence. The value 1 represents a package that is as far from the main sequence as is possible.

$$D' = |A + I - 1|$$

Applying the Metrics to the Payroll Application

Table 22-1 shows how the classes in the payroll model have been allocated to packages. Figure 22-7 shows the package diagram for the payroll application with all the metrics calculated. And Table 22-2 shows all of the metrics calculated for each package.

3. One might think that a better formula for A is the ratio of pure virtual functions to total member functions within the package. However, I have found that this formula weakens the abstraction metric too much. Even one pure virtual function will make a class abstract, and the power of that abstraction is more significant than the fact that the class may have dozens of concrete functions, especially when the DIP is being followed.

4. It is impossible to plot any package outside the unit square on the graph of A vs. I. This is because neither A nor I can exceed 1. The main sequence bisects this square from (0,1) to (1,0). The points within the square that are farthest from the main sequence are the two corners (0,0) and (1,1). Their distance from the main sequence is

$$\frac{\sqrt{2}}{2} = 0.70710678\ldots$$

Each package dependency in Figure 22-7 is adorned with two numbers. The number closest to the depender package represents the number of classes in that package that depend on the dependee package. The number closest to the dependee package represents the number of classes in that package that the depender package depends on.

Table 22-1

Package	Classes in Package		
Affiliations	ServiceCharge	UnionAffiliation	
AffiliationTransactions	ChangeAffiliationTransaction	ChangeUnaffiliated-Transaction	ChangeMember-Transaction
	ServiceChargeTransaction		
Application	Application		
Classifications	CommissionedClassification	HourlyClassification	SalariedClassification
	SalesReceipt	Timecard	
ClassificationTransaction	ChangeClassification-Transaction	ChangeCommissioned-Transaction	ChangeHourly-Transaction
	ChangeSalariedTransaction	SalesReceiptTransaction	TimecardTransaction
GeneralTransactions	AddCommissionedEmployee	AddEmployeeTransaction	AddHourlyEmployee
	AddSalariedEmployee	ChangeAddressTransaction	ChangeEmployee-Transaction
	ChangeNameTransaction	DeleteEmployeeTransaction	PaydayTransaction
Methods	DirectMethod	HoldMethod	MailMethod
MethodTransactions	ChangeDirectTransaction	ChangeHoldTransaction	ChangeMailTransaction
	ChangeMethodTransaction		
PayrollApplication	PayrollApplication		
PayrollDatabase	PayrollDatabase		
PayrollDatabase-Implementation	PayrollDatabase-Implementation		
PayrollDomain	Affiliation	Employee	PaymentClassification
	PaymentMethod	PaymentSchedule	
Schedules	BiweeklySchedule	MonthlySchedule	WeeklySchedule
TextParserTransaction-Source	TextParserTransactionSource		
TransactionApplication	TransactionApplication	Transaction	TransactionSource

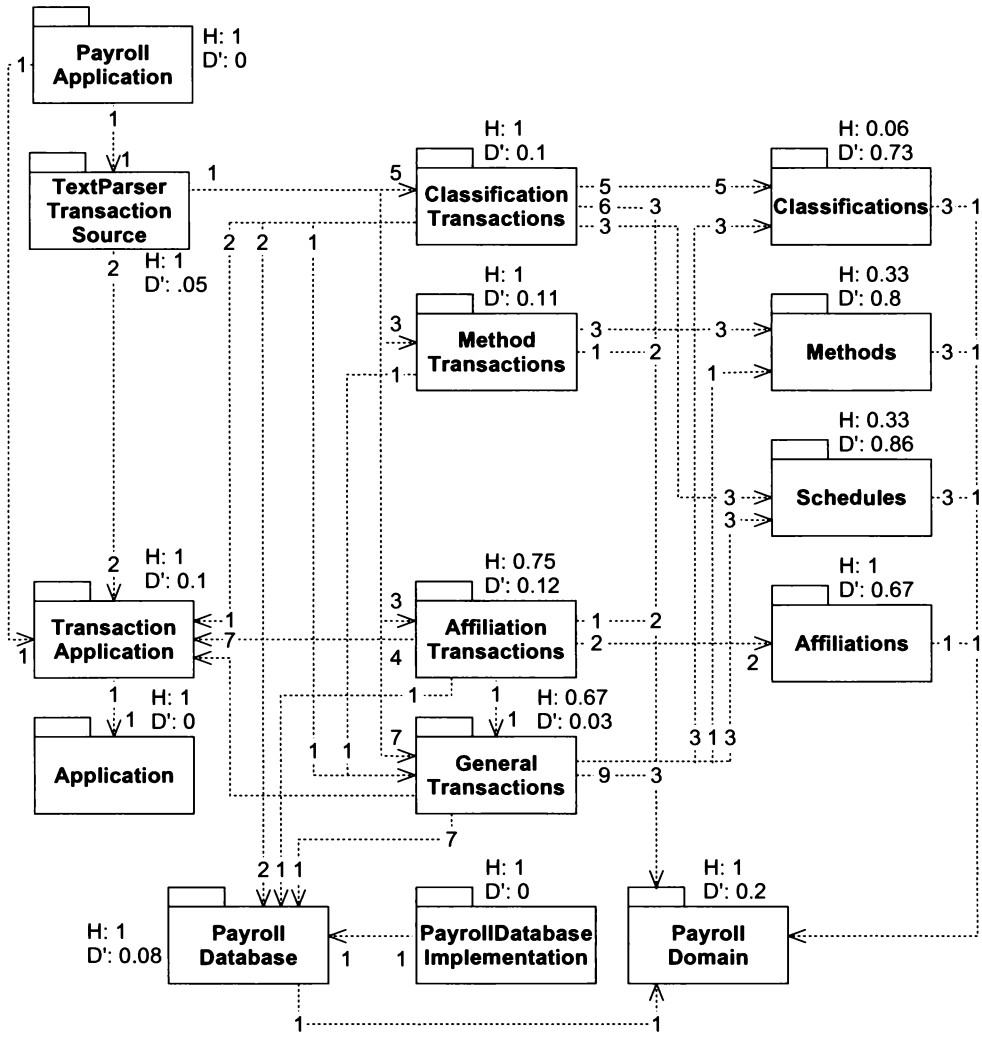


Figure 22-7 Package Diagram with Metrics

Each package in Figure 22-7 is adorned with the metrics that apply to it. Many of these metrics are encouraging. `PayrollApplication`, `PayrollDomain`, and `PayrollDatabase`, for example, have high relational cohesion and are either on or close to the main sequence. However, the `Classifications`, `Methods`, and `Schedules` packages show generally poor relational cohesion and are almost as far from the main sequence as is possible!

These numbers tell us that the partitioning of the classes into packages is weak. If we don't find a way to improve the numbers, then the development environment will be sensitive to change, which may cause unnecessary rerelease and retesting. Specifically, we have low-abstraction packages like `ClassificationTransactions` depending heavily on other low-abstraction packages like `Classifications`. Classes with low abstraction contain most of the detailed code and are therefore likely to change, which will force rerelease of the packages that depend on them. Thus the `ClassificationTransactions` package will have a very high release rate since it is subject to both its own high change rate and that of `Classifications`. As much as possible, we would like to limit the sensitivity of our development environment to change.

Clearly, if we have only two or three developers, they will be able to manage the development environment "in their heads," and the need to maintain packages on the main sequence, for this purpose, will not be great. The

Table 22-2

Package Name	N	A	Ca	Ce	R	H	I	A	D	D′
Affiliations	2	0	2	1	1	1	.33	0	.47	.67
AffiliationTransactions	4	1	1	7	2	.75	.88	.25	.09	.12
Application	1	1	1	0	0	1	0	1	0	0
Classifications	5	0	8	3	2	.06	.27	0	.51	.73
ClassificationTransaction	6	1	1	14	5	1	.93	.17	.07	.10
GeneralTransactions	9	2	4	12	5	.67	.75	.22	.02	.03
Methods	3	0	4	1	0	.33	.20	0	.57	.80
MethodTransactions	4	1	1	6	3	1	.86	.25	.08	.11
PayrollApplication	1	0	0	2	0	1	1	0	0	0
PayrollDatabase	1	1	11	1	0	1	.08	1	.06	.08
PayrollDatabaseImpl...	1	0	0	1	0	1	1	0	0	0
PayrollDomain	5	4	26	0	4	1	0	.80	.14	.20
Schedules	3	0	6	1	0	.33	.14	0	.61	.86
TextParserTransactionSource	1	0	1	20	0	1	.95	0	.03	.05
TransactionApplication	3	3	9	1	2	1	.1	1	.07	.10

more developers there are, however, the more difficult it is to keep the development environment sane. Moreover, the work required to obtain these metrics is minimal compared to the work required to do even a single retest and rerelease.[5] Therefore, it is a judgment call as to whether the work of computing these metrics will be a short-term loss or gain.

Object Factories

`Classifications` and `ClassificationTransactions` are so heavily depended on because the classes within them must be instantiated. For example, the `TextParserTransactionSource` class must be able to create `AddHourlyEmployeeTransaction` objects; thus, there is an afferent coupling from the `TextParserTransactionSource` package to the `ClassificationTransactions` package. Also, the `ChangeHourlyTransaction` class must be able to create `HourlyClassification` objects, so there is an afferent coupling from the `ClassificationTransactions` package to the `Classification` package.

Almost every other use of the objects within these packages is through their abstract interface. Were it not for the need to create each concrete object, the afferent couplings on these packages would not exist. For example, if `TextParserTransactionSource` did not need to create the different transactions, it would not depend on the four packages containing the transaction implementations.

This problem can be significantly mitigated by using the FACTORY pattern. Each package provides an object factory that is responsible for creating all the public objects within that package.

5. I spent about two hours compiling by hand the statistics and computing the metrics for the payroll example. Had I used one of the commonly available tools, it would have taken virtually no time at all.

The Object Factory for `TransactionImplementation`

Figure 22-8 shows how to build an object factory for the `TransactionImplementation` package. The `TransactionFactory` package contains the abstract base class, which defines the pure virtual functions that represent the constructors for the concrete transaction objects. The `TransactionImplementation` package contains the concrete derivative of the `TransactionFactory` class and uses all the concrete transactions in order to create them.

The `TransactionFactory` class has a static member declared as a `TransactionFactory` pointer. This member must be initialized by the main program to point to an instance of the concrete `TransactionFactoryImplementation` object.

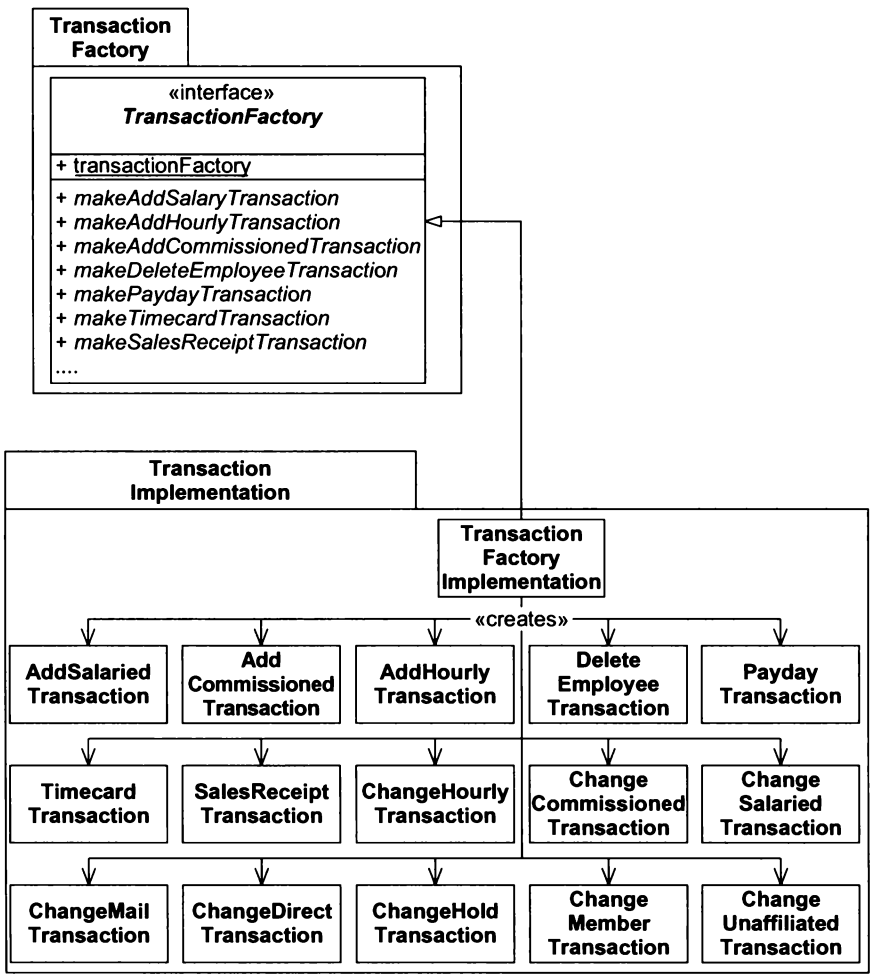


Figure 22-8 Object Factory for Transactions

Initializing the Factories

In order to create objects using the object factories, the static members of the abstract object factories must be initialized to point to the appropriate concrete factory. This must be done before any user attempts to use the factory. The best place to do this is usually the main program, which means that the main program depends on all the factories *and* on all the concrete packages. Thus, each concrete package will have at least one afferent coupling from the main program. This will force the concrete package off the main sequence a bit, but it cannot

be helped.[6] It means that we must rerelease the main program every time we change any of the concrete packages. Of course we should probably rerelease the main program for each change anyway, since it will need to be tested regardless.

Figures 22-9 and 22-10 show the static and dynamic structure of the main program in relation to the object factories.

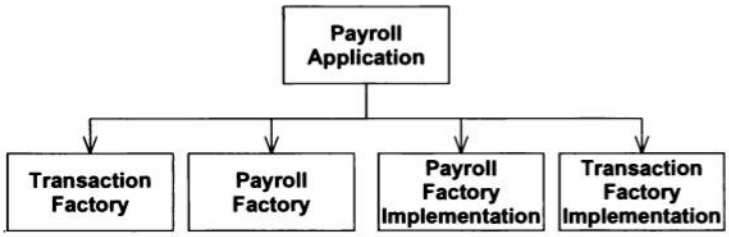


Figure 22-9 Static Structure of Main Program and Object Factories

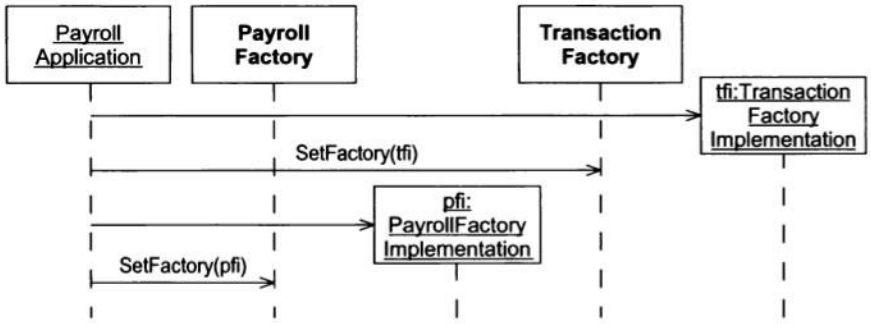


Figure 22-10 Dynamic Structure of Main Program and Object Factories

Rethinking the Cohesion Boundaries

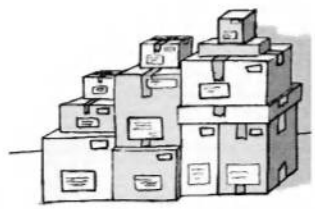
We initially separated `Classifications`, `Methods`, `Schedules`, and `Affiliations` in Figure 22-1. At the time, it seemed like a reasonable partitioning. After all, other users may want to reuse our schedule classes without reusing our affiliation classes. This partitioning was maintained after we split out the transactions into their own packages, creating a dual hierarchy. Perhaps this was too much. The diagram in Figure 22-7 is very tangled.

A tangled package diagram makes the management of releases difficult if it is done by hand. Although package diagrams would work well with an automated project-planning tool, most of us don't have that luxury. Thus, we need to keep our package diagrams as simple as is practical.

In my view, the transaction partitioning is more important than the functional partitioning. Thus, we will merge the transactions into a single `TransactionImplementation` package in Figure 22-11. We will also merge the `Classifications`, `Schedules`, `Methods`, and `Affiliations` packages into a single `PayrollImplementation` package.

The Final Package Structure

Table 22-3 shows the final allocation of classes to class package. Table 22-4 contains the metrics spreadsheet. Figure 22-11 shows the final package structure, which employs object factories to bring the concrete packages near the main sequence



6. As a practical solution, I usually ignore couplings from the main program.

Table 22-3

Packages	Classes in Packages		
AbstractTransactions	AddEmployeeTransaction	ChangeAffiliationTransaction	ChangeEmployee-Transaction
	ChangeClassification-Transaction	ChangeMethodTransaction	
Application	Application		
PayrollApplication	PayrollApplication		
PayrollDatabase	PayrollDatabase		
PayrollDatabaseImple-mentation	PayrollDatabase-Implementation		
PayrollDomain	Affiliation	Employee	PaymentClassification
	PaymentMethod	PaymentSchedule	
PayrollFactory	PayrollFactory		
PayrollImplementation	BiweeklySchedule	CommissionedClassification	DirectMethod
	HoldMethod	HourlyClassification	MailMethod
	MonthlySchedule	PayrollFactory-Implementation	SalariedClassification
	SalesReceipt	ServiceCharge	Timecard
	UnionAffiliation	WeeklySchedule	
TextParser-TransactionSource	TextParserTransactionSource		
Transaction-Application	Transaction	TransactionApplication	TransactionSource
TransactionFactory	TransactionFactory		
Transaction-Implementation	AddCommissionedEmployee	AddHourlyEmployee	AddSalariedEmployee
	ChangeAddressTransaction	ChangeCommissioned-Transaction	ChangeDirectTransaction
	ChangeHoldTransaction	ChangeHourlyTransaction	ChangeMailTransaction
	ChangeMemberTransaction	ChangeNameTransaction	ChangeSalariedTransaction
	ChangeUnaffiliatedTransaction	DeleteEmployee	PaydayTransaction
	SalesReceiptTransaction	ServiceChargeTransaction	TimecardTransaction
	TransactionFactory-Implementation		

Table 22-4

Package Name	N	A	Ca	Ce	R	H	I	A	D	D′
AbstractTransactions	5	5	13	1	0	.20	.07	1	.05	.07
Application	1	1	1	0	0	1	0	1	0	0
PayrollApplication	1	0	0	5	0	1	1	0	0	0
PayrollDatabase	1	1	19	5	0	1	.21	1	.15	.21
PayrollDatabaseImpl...	1	0	0	1	0	1	1	0	0	0
PayrollDomain	5	4	30	0	4	1	0	.80	.14	.20
PayrollFactory	1	1	12	4	0	1	.25	1	.18	.25
PayrollImplementation	14	0	1	5	3	.29	.83	0	.12	.17
TextParserTransactionSource	1	0	1	3	0	1	.75	0	.18	.25
TransactionApplication	3	3	14	1	3	1.33	.07	1	.05	.07
TransactionFactory	1	1	3	1	0	1	.25	1	.18	.25
TransactionImplementation	19	0	1	14	0	.05	.93	0	.05	.07

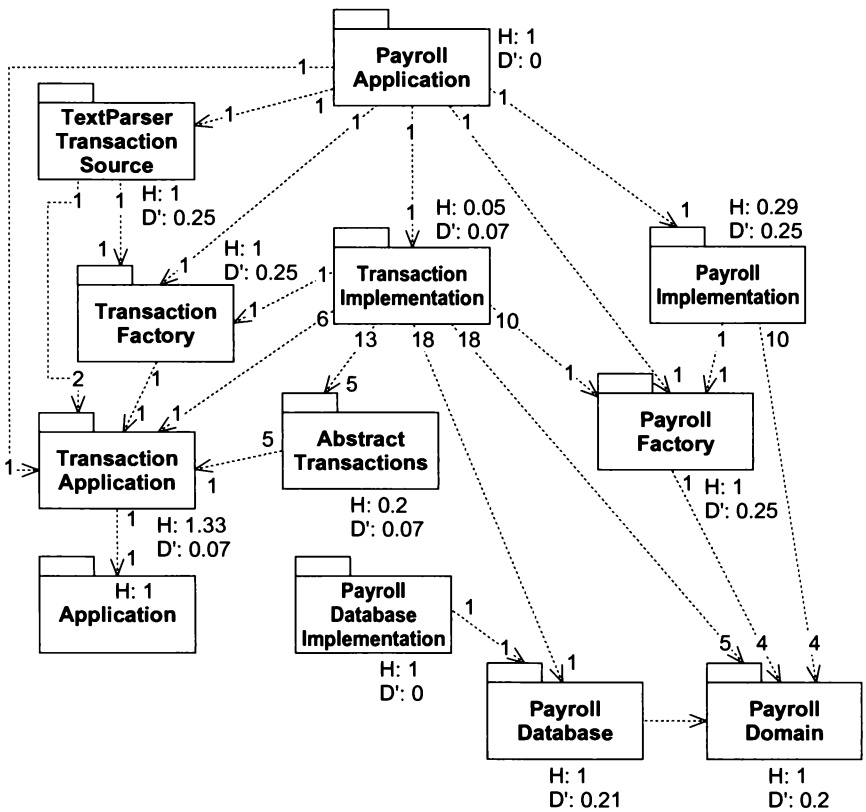


Figure 22-11 Final Payroll Package Structure

The metrics on this diagram are heartening. The relational cohesions are all very high (thanks in part to the relationships of the concrete factories to the objects that they create), and there are no significant deviations from the main sequence. Thus, the couplings between our packages are appropriate to a sane development environment. Our abstract packages are closed, reusable, and heavily depended on, while having few dependencies of their own. Our concrete packages are segregated on the basis of reuse, are heavily dependent on the abstract packages, and are not heavily depended on themselves.

Conclusion

The need to manage package structures is a function of the size of the program and the size of the development team. Even small teams need to partition the source code so that they can stay out of each other's way. Large programs can become opaque masses of source files without some kind of partitioning structure.

Bibliography

1. Benjamin/Cummings. *Object-Oriented Analysis and Design with Applications*, 2d ed.,1994.
2. DeMarco, Tom. *Controlling Software Projects*. Yourdon Press, 1982.

SECTION 5

The Weather Station Case Study

The following chapters contain an in-depth case study of a simple weather monitoring system. Although this case study is fictitious, it has nevertheless been constructed with a high degree of realism. We will encounter the problems of time pressure, legacy code, poor and mutating specifications, new untried technologies, etc. Our goal is to demonstrate how the principles, patterns, and practices that we have learned are used in the real world of software engineering.

As before, we will be encountering several useful design patterns while exploring the development of the weather station. The chapters leading up to the case study will describe those patterns.

23

COMPOSITE



The COMPOSITE pattern is a very simple pattern that has significant implications. The fundamental structure of the COMPOSITE pattern is shown in Figure 23-1. Here we see a hierarchy based on shapes. The `Shape` base class has two derivative shapes named `Circle` and `Square`. The third derivative is the composite. `CompositeShape` keeps a list of many `Shape` instances. When `draw()` is called on `CompositeShape`, it delegates that method to all the `Shape` instances in the list.

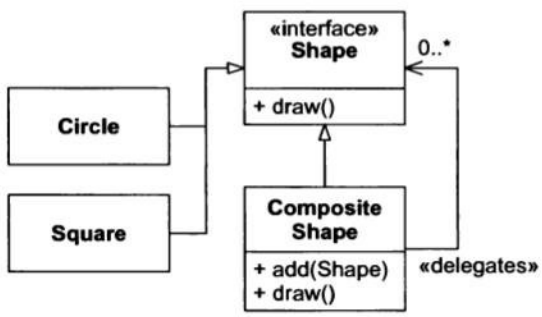


Figure 23-1 Composite Pattern

Thus, an instance of `CompositeShape` appears to the system to be a single `Shape`. It can be passed to any function or object that takes a `Shape`, and it will behave like a `Shape`. However, it is really a proxy[1] for a group of `Shape` instances.

Listing 23-1 and Listing 23-2 show one possible implementation of `CompositeShape`.

Listing 23-1

`Shape.java`

```
public interface Shape
{
  public void draw();
}
```

Listing 23-2

`CompositeShape.java`

```
import java.util.Vector;

public class CompositeShape implements Shape
{
  private Vector itsShapes = new Vector();
  public void add(Shape s)
  {
    itsShapes.add(s);
  }

  public void draw()
  {
    for (int i = 0; i < itsShapes.size(); i++)
    {
      Shape shape = (Shape) itsShapes.elementAt(i);
      shape.draw();
    }
  }
}
```

Example: Composite Commands

Consider the discussion of `Sensors` and `Command` objects we had back on page 152. Figure 13-3 showed a `Sensor` class using a `Command` class. When the `Sensor` detected its stimulus, it called `do()` on the `Command`.

What I failed to mention in that discussion was that there were often cases when a `Sensor` had to execute more than one `Command`. For example, when the paper reached a certain point in the paper path, it would trip an optical sensor. That sensor then stopped a motor, started another, and engaged a particular clutch.

At first we took this to mean that every `Sensor` class would have to maintain a list of `Command` objects. (See Figure 23-2.) However, we soon recognized that whenever a `Sensor` needed to execute more than one `Command`, it always treated those `Command` objects identically. That is, it just iterated over the list and called `do()` on each `Command`. This was ideal for the COMPOSITE pattern.

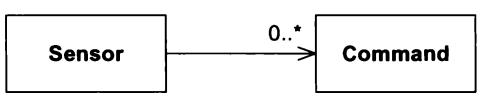


Figure 23-2 Sensor containing many Commands

1. Notice the similarity in structure to the PROXY pattern.

So we left the `Sensor` class alone and created a `CompositeCommand` as shown in Figure 23-3.

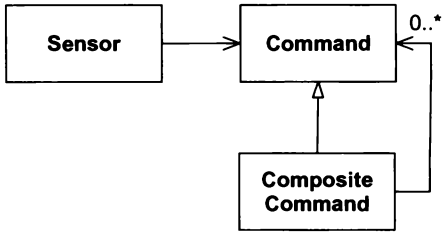


Figure 23-3 Composite Command

This meant that we didn't have to change the `Sensor` or the `Command`. We were able to add the plurality of `Commands` to a `Sensor` without changing either. This is an application of the OCP.

Multiplicity or Not Multiplicity

This leads to an interesting issue. We were able to make our `Sensors` behave as though they contained many `Commands`, without having to modify the `Sensors`. There must be many other situations like this in normal software design. There must be times when you could use COMPOSITE rather than building a list or vector of objects.

Let me say this a different way. The association between `Sensor` and `Command` is one-to-one. We were tempted to change that association to one to many. But instead we found a way to get one-to-many behavior, without a one-to-many relationship. A one-to-one relationship is much easier to understand, code, and maintain than a one-to-many relationship is; so this was clearly the right design trade-off. How many of the one-to-many relationships in your current project could be one-to-one if you used COMPOSITE?

Of course, not all one-to-many relationships can be reverted to one-to-one by using COMPOSITE. Only those in which every object in the list is treated identically are candidates. For example, if you maintained a list of employees and searched through that list for employees whose pay date is today, you probably shouldn't use the COMPOSITE pattern because you wouldn't be treating all the employees identically.

Still, there are quite a few one-to-many relationships that qualify for conversion to COMPOSITE. And the advantages are significant. Instead of duplicating the list management and iteration code in each of the clients, that code appears only once in the composite class.

24

OBSERVER—Backing into a Pattern



This chapter serves a special purpose. In it, I will describe the OBSERVER[1] pattern, but that is a minor objective. The primary objective of this chapter is to give you a demonstration of how your design and code can evolve to use a pattern.

In the preceding chapters, we have made use of many patterns. Often we presented them as a *fait accompli*, without showing how the code evolved to use the pattern. This might give you the idea that patterns are simply something you insert into your code and designs in completed form. This is not what I advise. Rather, I prefer to evolve the code I am working on in the direction of its needs. As I refactor it to resolve issues of coupling, simplicity, and expressiveness, I may find that the code has come close to a particular pattern. When that happens, I change the names of the classes and variables to use the name of the pattern, and change the structure of the code to use the pattern in a more regular form. Thus the code *backs into the pattern*.

This chapter sets up a simple problem and then shows how the design and code evolve to solve that problem. The result of the evolution winds up being the OBSERVER pattern. At each stage of the evolution, I will describe the issues I'm trying to resolve, and then show the steps that resolve them.

The Digital Clock

We have a clock object. This object catches millisecond interrupts (known as tics) from the operating system and turns them into the time of day. This object knows how to calculate seconds from milliseconds, minutes from seconds, hours from minutes, days from hours, etc. It knows how many days are in a month, and how many months are in a year. It knows all about leap years, when to have them, and when not. It knows about time. (See Figure 24-1.)

1. [GOF95], p. 293.

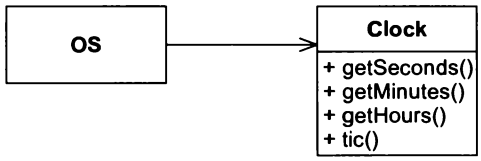


Figure 24-1 Clock

We'd like to create a digital clock that sits on our desktop and continuously displays the time of day. What is the simplest way to accomplish this? We could write the following code:

```
public void DisplayTime
{
  while(1)
  {
    int sec = clock.getSeconds();
    int min = clock.getMinutes();
    int hour = clock.getHours();
    showTime(hour,min,sec);
  }
}
```

Clearly this is suboptimal. It consumes all available CPU cycles to repeatedly display the time. Most of those displays will be wasted because the time will not have changed. It may be that this solution would be adequate in a digital watch or a digital wall clock since conserving CPU cycles is not very important in those systems. However, we don't want this CPU hog running on our desktop.

The fundamental problem is how to efficiently get data from the `Clock` to the `DigitalClock`. I'm going to assume that the `Clock` object and the `DigitalClock` object both exist. My interest is in how to connect them. I can test that connection simply by making sure that the data I get from the `Clock` are the same data I send to the `DigitalClock`.

A simple way to write this test is to create one interface that pretends to be the `Clock` and another that pretends to be the `DigitalClock`. Then I can write special test objects that implement those interfaces and verify that the connection between them works as expected. (See Figure 24-2.)

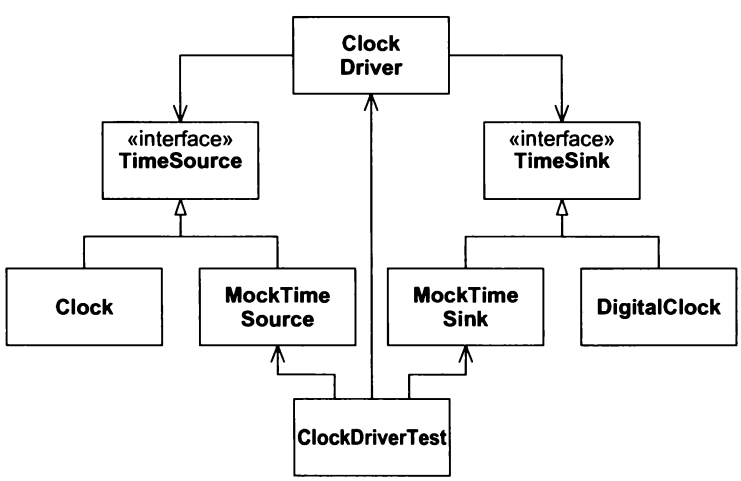


Figure 24-2 Testing the Digital Clock

The ClockDriverTest object will connect the ClockDriver to the two mock objects through the TimeSource and TimeSink interfaces. It will then check each of the mock objects to ensure that the ClockDriver manages to move the time from the source to the sink. If necessary, the ClockDriverTest will also ensure that efficiency is being conserved.

I think it's interesting that we have added interfaces to the design simply as a result of considering how to test it. In order to test a module, you have to be able to isolate it from the other modules in the system, just as we have isolated the ClockDriver from the Clock and DigitalClock. Considering tests first helps us to minimize the coupling in our designs.

OK, how does the ClockDriver work? Clearly, in order to be efficient, the ClockDriver must detect when the time in the TimeSource object has changed. Then, and only then, should it move the time to the TimeSink object. How can the ClockDriver know when the time has changed? It could poll the TimeSource, but that simply recreates the CPU hog problem.

The simplest way for the ClockDriver to know when the time has changed is for the Clock object to tell it. We could pass the ClockDriver to the Clock through the TimeSource interface and then, when the time changes, the Clock can update the ClockDriver. The ClockDriver will, in turn, set the time on the ClockSink. (See Figure 24-3.)

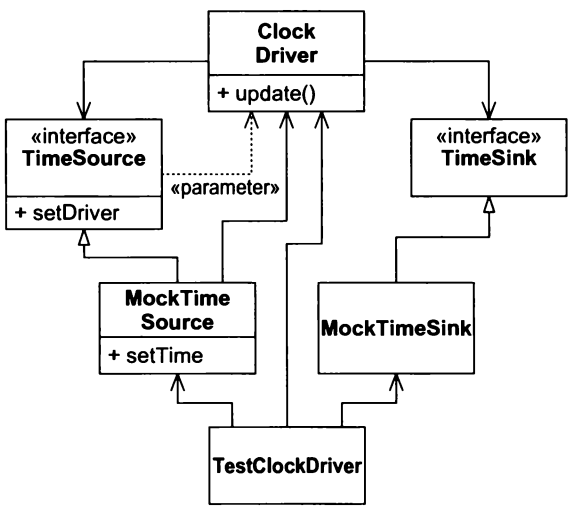


Figure 24-3 Getting the TimeSource to update the ClockDriver

Notice the dependency from the TimeSource to the ClockDriver. It is there because the argument to the setDriver method is a ClockDriver. I'm not very happy with this, since it implies that TimeSource objects must use ClockDriver objects in every case. However, I'll defer doing anything about the dependency until I get this program working.

Listing 24-1 shows the test case for the ClockDriver. Notice that it creates a ClockDriver and binds a MockTimeSource and a MockTimeSink to it. Then it sets the time in the source and expects the time to magically arrive at the sink. The rest of the code is shown in Listings 24-2 through 24-6.

Listing 24-1

ClockDriverTest.java

```
import junit.framework.*;

public class ClockDriverTest extends TestCase
{
  public ClockDriverTest(String name)
```

```
  {
    super(name);
  }

  public void testTimeChange()
  {
    MockTimeSource source = new MockTimeSource();
    MockTimeSink sink = new MockTimeSink();
    ClockDriver driver = new ClockDriver(source,sink);
    source.setTime(3,4,5);
    assertEquals(3, sink.getHours());
    assertEquals(4, sink.getMinutes());
    assertEquals(5, sink.getSeconds());

    source.setTime(7,8,9);
    assertEquals(7, sink.getHours());
    assertEquals(8, sink.getMinutes());
    assertEquals(9, sink.getSeconds());
  }
}
```

Listing 24-2

TimeSource.java

```
public interface TimeSource
{
  public void setDriver(ClockDriver driver);
}
```

Listing 24-3

TimeSink.java

```
public interface TimeSink
{
  public void setTime(int hours, int minutes, int seconds);
}
```

Listing 24-4

ClockDriver.java

```
public class ClockDriver
{
  private TimeSink itsSink;

  public ClockDriver(TimeSource source, TimeSink sink)
  {
    source.setDriver(this);
    itsSink = sink;
  }

  public void update(int hours, int minutes, int seconds)
  {
    itsSink.setTime(hours, minutes, seconds);
  }
}
```

Listing 24-5

MockTimeSource.java

```
public class MockTimeSource implements TimeSource
{
  private ClockDriver itsDriver;

  public void setTime(int hours, int minutes, int seconds)
  {
    itsDriver.update(hours, minutes, seconds);
  }

  public void setDriver(ClockDriver driver)
  {
    itsDriver = driver;
  }
}
```

Listing 24-6

MockTimeSink.java

```
public class MockTimeSink implements TimeSink
{
  private int itsHours;
  private int itsMinutes;
  private int itsSeconds;

  public int getSeconds()
  {
    return itsSeconds;
  }

  public int getMinutes()
  {
    return itsMinutes;
  }

  public int getHours()
  {
    return itsHours;
  }

  public void setTime(int hours, int minutes, int seconds)
  {
    itsHours = hours;
    itsMinutes = minutes;
    itsSeconds = seconds;
  }
}
```

OK, now that it works, I can think about cleaning it up. I don't like the dependency from `TimeSource` to `ClockDriver` because I want the `TimeSource` interface to be usable by anybody, not just `ClockDriver` objects. We can fix this by creating an interface that `TimeSource` can use and that `ClockDriver` can implement. We'll call this interface `ClockObserver`. See Listings 24-7 through 24-10. The code in **bold** has changed.

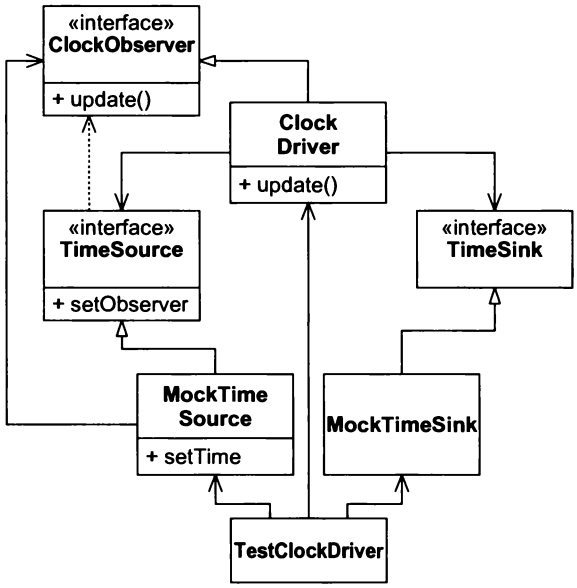


Figure 24-4 Breaking the depency of `TimeSource` upon `ClockDriver`

Listing 24-7

`ClockObserver.java`

```
public interface ClockObserver
{
  public void update(int hours, int minutes, int seconds);
}
```

Listing 24-8

`ClockDriver.java`

```
public class ClockDriver implements ClockObserver
{
  private TimeSink itsSink;

  public ClockDriver(TimeSource source, TimeSink sink)
  {
    source.setObserver(this);
    itsSink = sink;
  }

  public void update(int hours, int minutes, int seconds)
  {
    itsSink.setTime(hours, minutes, seconds);
  }
}
```

Listing 24-9

`TimeSource.java`

```
public interface TimeSource
{
  public void setObserver(ClockObserver observer);
}
```

Listing 24-10

MockTimeSource.java

```
public class MockTimeSource implements TimeSource
{
  private ClockObserver itsObserver;

  public void setTime(int hours, int minutes, int seconds)
  {
    itsObserver.update(hours, minutes, seconds);
  }

  public void setObserver(ClockObserver observer)
  {
    itsObserver = observer;
  }
}
```

This is better. Now anybody can make use of `TimeSource`. All they have to do is implement `ClockObserver` and call `SetObserver`, passing themselves in as the argument.

I'd like to be able to have more than one `TimeSink` getting the time. One might implement a digital clock. Another might be used to supply the time to a reminder service. Still another might start my nightly backup. In short, I'd like a single `TimeSource` to be able to supply the time to multiple `TimeSink` objects.

So I'll change the constructor of the `ClockDriver` to take just the `TimeSource` and then add a method named `addTimeSink` that allows you to add `TimeSink` instances any time you want.

The thing I don't like about this is that I now have two indirections. I have to tell the `TimeSource` who the `ClockObserver` is by calling `setObserver`, and I also have to tell the `ClockDriver` who the `TimeSink` instances are. Is this double indirection really necessary?

Looking at `ClockObserver` and `TimeSink`, I see that they both have the `setTime` method. It looks like `TimeSink` could implement `ClockObserver`. If I did this, then my test program could create a `MockTimeSink` and call `setObserver` on the `TimeSource`. I could get rid of the `ClockDriver` (and `TimeSink`) altogether! Listing 24-11 shows the changes to `ClockDriverTest`.

Listing 24-11

ClockDriverTest.java

```
import junit.framework.*;

public class ClockDriverTest extends TestCase
{
  public ClockDriverTest(String name)
  {
    super(name);
  }

  public void testTimeChange()
  {
    MockTimeSource source = new MockTimeSource();
    MockTimeSink sink = new MockTimeSink();
    source.setObserver(sink);

    source.setTime(3,4,5);
    assertEquals(3, sink.getHours());
```

```
    assertEquals(4, sink.getMinutes());
    assertEquals(5, sink.getSeconds());

    source.setTime(7,8,9);
    assertEquals(7, sink.getHours());
    assertEquals(8, sink.getMinutes());
    assertEquals(9, sink.getSeconds());
  }
}
```

This means that `MockTimeSink` should implement `ClockObserver` rather than `TimeSink`. See Listing 24-12. These changes work fine. Why did I think I needed a `ClockDriver` in the first place? Figure 24-5 shows the UML.

Listing 24-12

MockTimeSink.java

```
public class MockTimeSink implements ClockObserver
{
  private int itsHours;
  private int itsMinutes;
  private int itsSeconds;

  public int getSeconds()
  {
    return itsSeconds;
  }

  public int getMinutes()
  {
    return itsMinutes;
  }

  public int getHours()
  {
    return itsHours;
  }

  public void update(int hours, int minutes, int seconds)
  {
    itsHours = hours;
    itsMinutes = minutes;
    itsSeconds = seconds;
  }
}
```

Clearly this is much simpler.

OK, now we can handle multiple `TimeSink` objects by changing the `setObserver` function to `registerObserver` and by making sure that all the registered `ClockObserver` instances are held in a list and updated appropriately. This requires another change to the test program. Listing 24-13 shows the changes. I also did a little refactoring of the test program to make it smaller and easier to read.

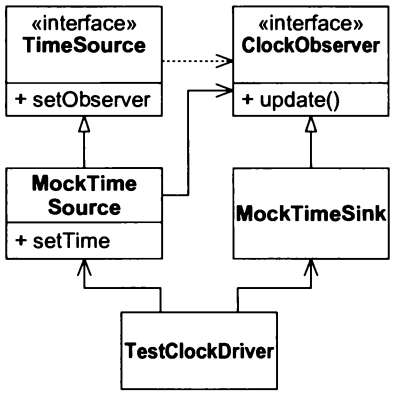


Figure 24-5 Removing `ClockDriver` and `TimeSink`

Listing 24-13

`ClockDriverTest.java`

```
import junit.framework.*;

public class ClockDriverTest extends TestCase
{
  private MockTimeSource source;
  private MockTimeSink sink;

  public ClockDriverTest(String name)
  {
    super(name);
  }

  public void setUp()
  {
    source = new MockTimeSource();
    sink = new  MockTimeSink();
    source.registerObserver(sink);
  }

    private void assertSinkEquals(
    MockTimeSink sink, int hours, int minutes, int seconds)
  {
    assertEquals(hours, sink.getHours());
    assertEquals(minutes, sink.getMinutes());
    assertEquals(seconds, sink.getSeconds());
  }

  public void testTimeChange()
  {
    source.setTime(3,4,5);
    assertSinkEquals(sink, 3,4,5);

    source.setTime(7,8,9);
    assertSinkEquals(sink, 7,8,9);
  }
```